AI VOICE BOT

A real-time voice-enabled AI chatbot that listens to user speech, processes it using an LLM API, and responds with voice output.

Overview:
This project is a full-stack AI assistant built using FastAPI and Web Speech API.
It allows users to interact with an AI system using voice input and receive intelligent spoken responses.

Requirement:

Hardware	Software
A virtual box – Linuxmint installed os with 10gb storage and 2gb ram	Python3,ApiKey,chrome browser(support audio)

Features:

▷ 🎤 Speech-to-Text (Voice Input)
▷ 🤖 AI Chat Response (OpenRouter API)
▷ 🔊 Text-to-Speech (Voice Output)
▷ 🔁 Continuous Listening Mode
▷ 🛑 Stop Conversation Anytime
▷ 🔄 Reset / Start New Chat
▷ 🌐 Works in Browser (No app install required)

Layer Technology:

Backend	FastAPI (Python)
Frontend	HTML, JavaScript
AI Model	OpenRouter API
Voice Input	Web Speech API
Voice Output	SpeechSynthesis API
Version Control	Git & GitHub

Architecture:

User Voice → Speech Recognition → API Call → AI Model → Response → Speech Output

Terminal - ubuntu@ubuntu-VirtualBox: ~/Desktop/ai-voice-bot

File Edit View Terminal Tabs Help

```
ubuntu@ubuntu-VirtualBox:~/Desktop/ai-voice-bot$ source venv/bin/activate
(venv) ubuntu@ubuntu-VirtualBox:~/Desktop/ai-voice-bot$
```

Terminal - ubuntu@ubuntu-VirtualBox: ~/Desktop/ai-voice-bot

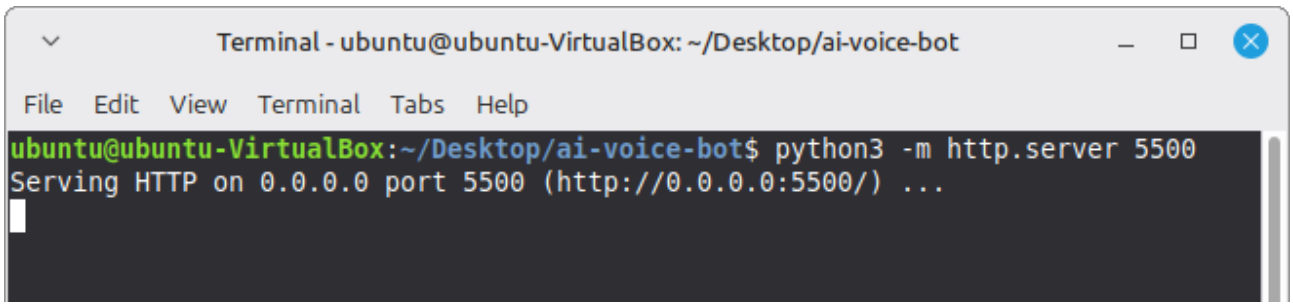
File Edit View Terminal Tabs Help

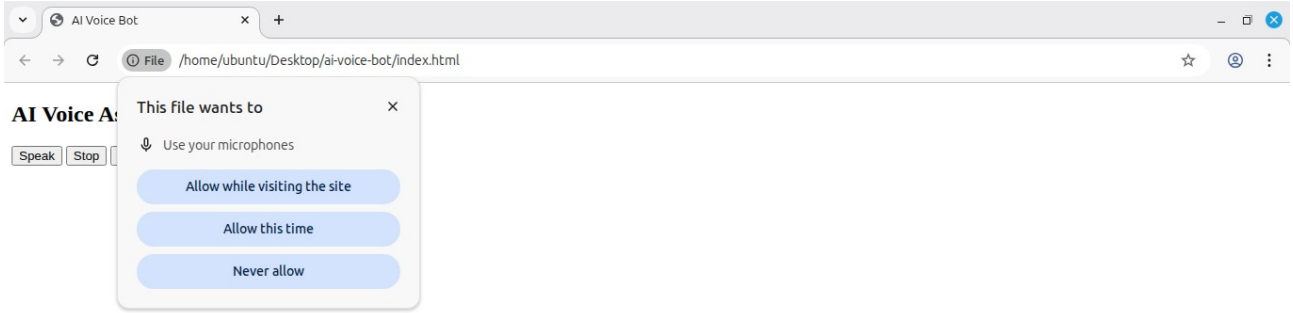
```
 create mode 100644 app.py
 create mode 100644 index.html
ubuntu@ubuntu-VirtualBox:~/Desktop/ai-voice-bot$ git branch -M main
ubuntu@ubuntu-VirtualBox:~/Desktop/ai-voice-bot$ git remote add origin https://g
ithub.com/prabuSR/ai-voice-bot.git
ubuntu@ubuntu-VirtualBox:~/Desktop/ai-voice-bot$ git push -u origin main
Username for 'https://github.com': prabuSR
Password for 'https://prabuSR@github.com':
remote: Invalid username or token. Password authentication is not supported for
Git operations.
fatal: Authentication failed for 'https://github.com/prabuSR/ai-voice-bot.git/'
ubuntu@ubuntu-VirtualBox:~/Desktop/ai-voice-bot$ git push -u origin main
Username for 'https://github.com': prabuSR
Password for 'https://prabuSR@github.com':
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 2 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 1.75 KiB | 449.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/prabuSR/ai-voice-bot.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
ubuntu@ubuntu-VirtualBox:~/Desktop/ai-voice-bot$
```

Terminal - ubuntu@ubuntu-VirtualBox: ~/Desktop/ai-voice-bot

File Edit View Terminal Tabs Help

```
ubuntu@ubuntu-VirtualBox:~/Desktop/ai-voice-bot$ source venv/bin/activate
(venv) ubuntu@ubuntu-VirtualBox:~/Desktop/ai-voice-bot$ uvicorn app:app --reload --port 8001
INFO:     Will watch for changes in these directories: ['/home/ubuntu/Desktop/ai-voice-bot']
INFO:     Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)
INFO:     Started reloader process [2862] using StatReload
INFO:     Started server process [2864]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```





Installation & Setup

1.Install python version and necessary packages

sudo apt install python3 python3-pip
python3 --version
pip3 –version

2. Create project folder

mkdir ai-voice-bot
cd ai-voice-bot
python3 -m venv venv
source venv/bin/activate

3. Install Backend packages

pip install fastapi uvicorn openai python-dotenv gtts

4. Create Basic Chat API - **app.py**

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
import requests

app = FastAPI()
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],  # allow all (for development)
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
API_KEY = "sk-or-v1-
583e6ac55365e2415ce1bbf8a2e063437d15132487ad1168bd7aafa6f9d699b2"

class Message(BaseModel):
    message: str

@app.post("/chat")
async def chat(data: Message):
    try:
        url = "https://openrouter.ai/api/v1/chat/completions"

        headers = {
            "Authorization": f"Bearer {API_KEY}",
            "Content-Type": "application/json"
        }

        payload = {
            "model": "openrouter/auto",
            "messages": [
                {"role": "system", "content": "You are a friendly motivational assistant."},
                {"role": "user", "content": data.message}
            ]
        }
        response = requests.post(url, headers=headers, json=payload)

        result = response.json()

        return {
            "reply": result["choices"][0]["message"]["content"]
        }
    except Exception as e:
return {"reply": "Error: " + str(e)}
```

5. Install Required Libraries

```
pip install fastapi uvicorn openai python-dotenv
```

6. Add Your API Key (secret do not share or leak)

```
nano .env
OPENAI_API_KEY=your_api_key_here(sk-xxxxx)
```

7. run backend

```
uvicorn app:app --reload --port 8001
```

8. create a frontend html

```
<!DOCTYPE html>
<html>
<head>
   <title>AI Voice Bot</title>
</head>
<body>

<h2>AI Voice Assistant</h2>

<button onclick="startListening()">Speak</button>
<button onclick="stopListening()">Stop</button>
<button onclick="resetChat()">New Chat</button>

<p id="userText"></p>
<p id="botReply"></p>

<script>
const apiUrl = "http://localhost:8001/chat";

let recognition;
let isListening = false;

function startListening() {
   const SpeechRecognition = window.SpeechRecognition || window.webkitSpeechRecognition;

   if (!SpeechRecognition) {
      alert("Speech Recognition not supported");
      return;
   }

   recognition = new SpeechRecognition();
   recognition.lang = "en-US";
   recognition.continuous = true;

   recognition.onstart = function() {
      isListening = true;
      console.log("Listening...");
   };
```

```
    recognition.onresult = function(event) {
        const text = event.results[event.results.length - 1][0].transcript;
        document.getElementById("userText").innerText = "You: " + text;

        sendToAPI(text);
    };

    recognition.onerror = function(event) {
        console.log("Error:", event.error);
    };

    recognition.start();
}

function stopListening() {
    if (recognition && isListening) {
        recognition.stop();
        isListening = false;
        console.log("Stopped listening");
    }

    // stop voice also
    speechSynthesis.cancel();
}

function resetChat() {
    document.getElementById("userText").innerText = "";
    document.getElementById("botReply").innerText = "";

    // stop everything
    if (recognition) recognition.stop();
    speechSynthesis.cancel();

    console.log("Chat reset");
}

function sendToAPI(message) {
    fetch(apiUrl, {
        method: "POST",
        headers: {
            "Content-Type": "application/json"
        },
        body: JSON.stringify({ message: message })
    })
    .then(res => res.json())
    .then(data => {
        document.getElementById("botReply").innerText = "Bot: " + data.reply;

        speechSynthesis.cancel();

        setTimeout(() => {
```

```
            const speech = new SpeechSynthesisUtterance(data.reply);
            speechSynthesis.speak(speech);
        }, 200);
    });
}
</script>

</body>
</html>
```

9. run frontend

python3 -m http.server 5500
browser:- **http://localhost:5500**

How It Works

1. Click 🎤 Start
2. Speak into microphone
3. Voice is converted to text
4. Text is sent to backend API
5. AI generates response
6. Response is spoken back to user

Challenges Faced

- CORS issues between frontend and backend
- API authentication errors (401, 429)
- Model availability issues (OpenRouter)
- Browser speech recognition limitations (Linux)
- SpeechSynthesis stopping after first response

Solutions Implemented

- Enabled CORS in FastAPI
- Switched to free API (OpenRouter)
- Added fallback models
- Implemented speech reset using `speechSynthesis.cancel()`
- Added continuous listening mode

Conclusion:

Built a voice-enabled AI assistant using FastAPI, OpenRouter API, and Web Speech APIs for real-time speech-to-text and text-to-speech interaction.
Implemented end-to-end integration of frontend and backend with handling of CORS, API errors, and model limitations.
Demonstrated practical skills in full-stack development, API debugging, and building interactive AI applications.